Person 1: `gcs.py` (Ground Control Station, registration and certificates)

Your job in one line: before any drone flies, the GCS decides who is a legitimate drone and gives it a signed certificate that proves it. You also handle captured drones (revocation).

Run your demo: `python gcs.py`

---

1. What to say (about 3 minutes)

> "Our protocol has three kinds of entities: the **Ground Control Station** (trusted authority), the **drones**, and the **attacker** who controls the radio channel.
>
> Before take-off, every drone is registered with the GCS over a secure, physical channel on the ground. The drone makes its own key pair. The private key never leaves the drone, and the drone gives its public key to the GCS. The GCS then issues a **certificate**: drone ID + public key + expiry time, signed with the GCS's private key using **Ed25519 digital signatures**.
>
> Every drone also stores the GCS's public key. In the air there is no internet and no server to contact, so a drone checks a stranger's certificate offline using only that stored key. If the signature is valid, the GCS vouched for this drone.
>
> If a drone is captured, the GCS puts its ID on the **revocation list**, and honest drones refuse it from then on."

Then run `python gcs.py` and read out the 5 lines: issued, accepted, edited cert rejected, fake GCS rejected, revoked drone rejected.

---

2. Libraries in this file

Import	Where it's from	Why we need it
`json`	Python standard library	Turns a dict into text/bytes, used by `ser()`
`time`	Python standard library	Current time, used for certificate expiry
`Ed25519PrivateKey`, `Ed25519PublicKey`	`cryptography` (pyca)	Ed25519 digital signatures: make a key, sign, verify
`serialization`	`cryptography`	Export a public key as raw 32 bytes
`InvalidSignature`	`cryptography`	The exception thrown when a signature check fails

`cryptography` is the standard, audited Python crypto library (it wraps OpenSSL). We never write our own crypto maths. That's the first rule of secure protocol design: you design the *protocol*, not the primitives.

---

3. Line by line

```
class AuthError(Exception):
    """Raised by every file whenever a security check fails."""
```

Our own error type. Every rejection in the whole project (bad cert, replay, tamper) raises `AuthError`. That's how `attacks.py` knows an attack was *blocked*. It's defined here because `gcs.py` is the base file that every other file imports.

```
def ser(d: dict) -> bytes:
    return json.dumps(d, sort_keys=True, separators=(",", ":")).encode()
```

A signature is computed over **bytes**, not over a Python dict. If the signer produced `{"id":..,"pk":..}` and the verifier produced `{"pk":..,"id":..}`, the bytes would differ and a valid signature would fail. `sort_keys=True` fixes the key order, `separators` removes spaces, and `.encode()` turns the text into bytes. This is called **canonical encoding**. `drone.py` and `session.py` also use it.

```
def pub_hex(private_key) -> str:
    raw = private_key.public_key().public_bytes(serialization.Encoding.Raw,
serialization.PublicFormat.Raw)
    return raw.hex()
```

Takes a private key, gets its public half, and exports it as 32 raw bytes, then as hex text so it fits in a JSON message. `drone.py` uses this for both Ed25519 keys and X25519 (DH) keys.

```
def verify_sig(pk_hex, sig_hex, data) -> bool:
    try:
        Ed25519PublicKey.from_public_bytes(bytes.fromhex(pk_hex)).verify(bytes.fromhex(sig_hex), data)
        return True
    except (InvalidSignature, ValueError):
        return False
```

Loads the public key from hex, then `verify()` checks the signature. The library *raises* on failure rather than returning False, so we catch:

- `InvalidSignature`: wrong signature, or the data was changed.
- `ValueError`: garbage hex or the wrong key length (attacker-crafted junk).

Returning `True` / `False` keeps the callers simple. `drone.py` uses this to check every handshake message.

```
class GCS:
    def __init__(self):
        self._sk = Ed25519PrivateKey.generate()
        self.pk = pub_hex(self._sk)
        self.revoked = set()
```

- `_sk` is the GCS **private** key. The leading `_` means "private, don't touch". It signs certificates and is the most important secret in the system.
- `pk` is the GCS **public** key. It's copied into every drone and is the *trust anchor*.
- `revoked` is the **CRL** (certificate revocation list). A `set` gives a fast "is this ID in it?" check.

```
    def register(self, drone_id, drone_pk, valid_secs=3600):
        body = {"id": drone_id, "pk": drone_pk, "exp": int(time.time()) + valid_secs}
        return {"body": body, "sig": self._sk.sign(ser(body)).hex()}
```

This is the **registration phase**. The certificate body binds three things: *who* (`id`), *their key* (`pk`) and *until when* (`exp`, 1 hour by default). The GCS signs the canonical bytes of the body. Changing any single character of the body afterwards makes the signature invalid. That's what makes the certificate unforgeable.

```
    def revoke(self, drone_id):
        self.revoked.add(drone_id)
```

Adds a captured or compromised drone to the CRL.

```
def verify_certificate(cert, gcs_pk, crl, claimed_id) -> str:
    if not verify_sig(gcs_pk, cert["sig"], ser(cert["body"])):
        raise AuthError("certificate not signed by GCS")
    if cert["body"]["exp"] < time.time():
        raise AuthError("certificate expired")
    if cert["body"]["id"] != claimed_id:
        raise AuthError("certificate does not match sender id")
    if claimed_id in crl:
        raise AuthError("drone revoked")
    return cert["body"]["pk"]
```

A drone runs this on every certificate it receives (it's called from `Drone._check` in `drone.py`). There are four checks, **in this order**:

1. **GCS signature first.** We trust *nothing* in the body until we know the GCS signed it. This also stops expiry editing: changing `exp` breaks the signature.
2. **Expiry.** Old certificates stop working, which limits the damage if a key leaks.
3. **ID matches the sender.** This stops "I'm A, here's B's certificate".
4. **Not revoked.** This is the **authorization** check: valid *and* still allowed.

It returns the peer's public key, which `drone.py` then uses to check the message signature.

```
if __name__ == "__main__":
```

This block only runs when you type `python gcs.py`, not when another file imports `gcs`. It's your live demo.

---

4. How this file links to the others

```
gcs.py ──(AuthError, ser)──────────────────────► session.py
gcs.py ──(GCS, pub_hex, verify_sig, verify_certificate)──► drone.py
```

- `drone.py` creates a drone with `Drone(id, gcs)`, which calls `gcs.register(...)`.
- `drone.py`'s `_check()` calls `verify_certificate(...)` and then `verify_sig(...)`.
- `gcs.py` imports **nothing** from our other files. It's the foundation.

---

5. Lecture-plan topics (CS428) and how to say them

Lecture topic	Where in `gcs.py`	How to explain it
Digital Signature Schemes	`register()` signs, `verify_sig()` verifies	"Only the GCS can produce a valid signature; anyone with the public key can check it."
Hardness assumption: Discrete logarithm problem	Ed25519 security	"Ed25519 works on an elliptic curve. Forging a signature means solving the elliptic-curve discrete log problem, which is computationally infeasible."
Computational Security	whole design	"We don't claim perfect secrecy. We claim an attacker with realistic computing power can't break it."
Zero knowledge proofs	Ed25519 is a Schnorr-type signature	"Schnorr signatures come from the Schnorr identification protocol, a zero-knowledge proof that you know the private key, made non-interactive with the Fiat–Shamir hash trick."
The Merkle–Damgård Transform	Ed25519 hashes with SHA-512 internally	"SHA-512 is a Merkle–Damgård hash."
SSL/TLS, session establishment	certificates	"Same idea as TLS certificates: a trusted CA (our GCS) vouches for a public key."

---

6. Likely questions

- **Why certificates instead of passwords?** Drones can't type passwords, and a shared password would let one captured drone impersonate all of them. With per-drone keys, a captured drone only exposes itself, and we revoke it.
- **Why Ed25519 and not RSA?** Small keys (32 bytes) and signatures (64 bytes), which matters on a radio link. It's fast on low-power hardware and has no padding pitfalls. RSA-3072 has about the same security level with much bigger keys.

- **Is the certificate secret?** No. It's public. Showing it proves nothing on its own. The drone must also *sign* with the matching private key (that's `drone.py`'s job, and the "stolen cert" attack test).
- **What if the GCS private key is stolen?** Then an attacker can make valid certificates. That's the single point of trust. As future work, split the GCS key with **Secret Sharing / Threshold signatures** (a lecture topic), so that no single GCS server holds the whole key.
- **How does the CRL reach drones in flight?** In the code, drones share the GCS's set object (simulation, marked with a `ponytail:` comment). In reality the GCS pushes a signed CRL before each mission, and the short 1-hour expiry limits the risk in between.
- **Non-repudiation?** Only the drone has its private key, so a valid signature proves that drone sent the message and it can't deny it later.